

Course Outline: Introduction to React and Next.js

Title: Introduction to React and Next.js — Building Component-Based Web Applications **Complexity:** Normal **Prerequisite:** Week 5 Day 14 — Memory Bank and AI Context Rules (Skill 11) **Target Audience:** Beginner developers who have completed HTML, CSS, Tailwind, and TypeScript fundamentals and are ready to enter structured frontend development **Total Lessons:** 22 **Format:** Mixed (32% hands-on — 7 lessons)

Course Description

Up until now, students have built static pages with HTML, styled them with CSS and Tailwind, and written logic with TypeScript. This learnpack marks the transition into modern frontend development: a world where user interfaces are built from reusable, self-contained pieces called components.

This course introduces React as a component model and Next.js as the framework that makes React production-ready. Students will learn how projects are organized, how navigation works through the App Router, and how the browser and server each play a role in rendering a page. By the end, students will have built their first multi-page Next.js application from components — laying the foundation for the deeper React work that follows in the coming days.

This is a first-contact course. Topics like props, state, and effects are introduced so students recognize them and understand their purpose — not to master them yet. Full treatment of those concepts comes in Skill 12.2.

Course Philosophy

This course emphasizes:

- **Structural thinking first:** Understand how a project is organized before writing any logic
 - **Path of least resistance:** Learn the simplest way to create a page, then add layers
 - **Context over memorization:** Understand why Next.js conventions exist, not just what they are
-

Course Statistics Summary

Section	Title	Lessons
00	Welcome	1
01	Web Components in React	3
02	Next.js — React's Framework	3
03	Project Structure and Organization	3
04	App Router and Navigation	4
05	Rendering Strategies	2
06	Your First Real Component in Next.js	3
07	Assessment	2
08	Code Challenge	1
Total		22

Section 00: Welcome

00.0 Welcome: From HTML Pages to Component-Based Apps 📖

Learning Objectives:

- Understand what problem React and Next.js solve compared to plain HTML/CSS
- Recognize how this course connects to previous skills and sets up what comes next
- Know what students will be able to build by the end of this learnpack

Content Outline:

- A brief recap of how far students have come: HTML structure, CSS styling, TypeScript logic
- The limitation of static HTML: duplicating code, no reuse, no dynamic content
- The core idea React introduces: building UIs from components instead of pages
- What Next.js adds on top: routing, project structure, server-side features
- What this course covers and what it intentionally defers to the next lesson

Transitions: This welcome lesson sets the stage. The next section unpacks the core concept React is built around: the web component.

Key Principles:

- Static HTML does not scale — components are the solution
- React is a UI library; Next.js is the full framework built around it
- This course is about understanding structure and concepts, not memorizing syntax

Examples:

- Imagine building a navigation bar that appears on 10 pages — in HTML you copy it 10 times; in React you write it once and reuse it everywhere
 - Think of a component like a custom HTML tag you design yourself: `<Navbar />`, `<Footer />`, `<ProductCard />`
-

Section 01: Web Components in React

01.0 What Is a Web Component in React? 📖

Learning Objectives:

- Define what a React component is in plain terms
- Understand the relationship between components and HTML output
- Distinguish between a component definition and a component instance

Content Outline:

- The concept of a component: a reusable, self-contained piece of UI
- How a React component maps to HTML — it returns markup
- The difference between defining a component (the blueprint) and using it (the instance)
- Functional components as the modern standard (React 16.8+)
- What a component file looks like: naming conventions, file extensions (`.jsx`, `.tsx`)
- The rule: component names must start with a capital letter

Transitions: Now that we know what a component is, the next lesson explains how React actually gets that component onto the screen — and why that matters.

Key Principles:

- A component is a function that returns UI

- Components are reusable: write once, use many times
- The capital letter naming rule is not optional — React uses it to distinguish components from HTML tags

Examples:

- A `<Button />` component that renders `<button className="...">Click me</button>`
 - A `<UserCard />` component that renders a styled card with a name and avatar
 - The same `<ProductCard />` component used 12 times in a product grid, each with different data
-

01.1 How React Updates the Web (The DOM Problem, Solved)

Learning Objectives:

- Explain what the DOM is and why direct DOM manipulation is problematic at scale
- Understand what the Virtual DOM is and how React uses it
- Recognize that React handles page updates automatically — students don't write `document.querySelector` in React

Content Outline:

- Quick recap: the DOM is the live, in-memory representation of the page
- The problem with manipulating the DOM directly: it's slow, error-prone, and hard to coordinate
- What React's Virtual DOM is: a lightweight copy of the DOM kept in memory
- How React diffs the Virtual DOM against the real DOM and applies only the necessary changes
- Why this matters: React handles updates — you just describe what the UI should look like, and React figures out how to get there
- The mental shift: from *imperative* ("find this element and change it") to *declarative* ("here is what the UI should look like")

Transitions: Now that we understand why React exists and how it updates the page, it's time to write our first component.

Key Principles:

- You describe the UI; React decides how to update the DOM
- Declarative thinking is the core mental model of React
- Direct DOM manipulation (`document.querySelector` , `innerHTML`) is an anti-pattern in React

Examples:

- Without React: `document.getElementById("count").innerText = 5` — you control every step
 - With React: you say "this number should be 5" and React handles the DOM update
 - The Virtual DOM diff in action: only the changed text node is updated, not the whole page
-

01.2 Creating Your First Component

Learning Objectives:

- Write a basic functional component that returns JSX
- Use a component inside another component
- Understand JSX syntax rules: single root element, `className` instead of `class`

Content Outline:

- Setting up: what a `.tsx` component file looks like
- Writing a functional component: the function signature and the return statement
- JSX rules students must know from day one:
 - Every component must return a single root element (or a fragment `<>`)
 - `class` becomes `className` in JSX
 - Self-closing tags must close: `<img />` not `<img>`
- Using one component inside another: composition

- Exporting a component for use elsewhere

Transitions: We now know how to write and compose components. The next section introduces Next.js — the framework that organizes these components into a full application.

Key Principles:

- JSX is not HTML — it compiles to JavaScript
- Components nest inside each other to build complex UIs
- Export your components so they can be used across the project

Examples:

- A `<Header />` component containing an `<h1>` and a `<nav>`
- A `<Page />` component that uses `<Header />`, `<Main />`, and `<Footer />`
- Fixing the common error: "JSX expressions must have one parent element"

Hands-On Exercise: Write three components from scratch: a `<Greeting />` that renders your name, a `<Badge />` that renders a styled label, and an `<App />` that composes both. Verify it renders correctly. Fix any JSX syntax errors that appear.

Success Criteria:

- All three components are written as functions that return JSX
 - `<App />` renders `<Greeting />` and `<Badge />` as children
 - No JSX syntax errors (single root, proper tag closing, `className` used)
-

Section 02: Next.js — React's Framework

02.0 What Is Next.js and Why Does It Exist?

Learning Objectives:

- Explain what problem Next.js solves that React alone does not
- List the main features Next.js adds on top of React
- Understand why 4Geeks teaches Next.js instead of plain React

Content Outline:

- React is a UI library, not a complete framework — it solves the "component" problem but leaves the rest to you
- What you'd have to set up manually without Next.js: routing, bundling, server rendering, image optimization
- Next.js as the opinionated framework: it makes these decisions for you
- Key Next.js features at a glance: file-based routing, server-side rendering, API routes, image optimization, built-in TypeScript support
- Why the industry uses Next.js: it's the most widely adopted React framework for production applications
- The 4Geeks stack decision: Next.js + TypeScript + Tailwind

Transitions: Now that we know what Next.js brings to the table, the next lesson clarifies the division of responsibility between React and Next.js.

Key Principles:

- React builds components; Next.js builds applications
- Convention over configuration: Next.js makes common decisions so you don't have to
- Learning Next.js is learning React — you write React components inside a Next.js project

Examples:

- Without Next.js: you'd need to install and configure React Router, Webpack or Vite, a server renderer — a full day's setup
- With Next.js: `npx create-next-app` and you have routing, SSR, TypeScript, and Tailwind configured in minutes

- Real-world usage: Vercel, Notion, Twitch, and many other production apps are built with Next.js
-

02.1 React vs Next.js — What Each One Does

Learning Objectives:

- Clearly articulate the difference between React and Next.js
- Identify which features belong to React and which belong to Next.js
- Avoid common confusion about the two (e.g., "I'm using Next instead of React")

Content Outline:

- React's responsibility: the component model, the Virtual DOM, state and effects (preview only)
- Next.js's responsibility: routing, rendering strategies, project structure, build tooling, deployment
- The key framing: Next.js *uses* React — you don't choose one or the other
- A side-by-side comparison: same `<Button />` component works in both
- When you might use React without Next.js: embedded widgets, React Native (mention only)

Transitions: With the distinction clear, we're ready to start a real Next.js project and see how it's organized.

Key Principles:

- Next.js is built on React, not a replacement for it
- All React knowledge applies inside Next.js projects
- Next.js adds the "application layer" — routing, rendering, project conventions

Examples:

- React = the engine; Next.js = the car
 - Writing `<Button />` is React; deciding which page `<Button />` appears on is Next.js
 - The `"use client"` directive: a Next.js concept that doesn't exist in plain React
-

02.2 Setting Up a Next.js Project

Learning Objectives:

- Scaffold a new Next.js project using `create-next-app`
- Identify the key files and folders generated automatically
- Run the development server and view the app in the browser

Content Outline:

- Running `npx create-next-app@latest` — what each prompt means
- The options students should choose: TypeScript yes, Tailwind yes, App Router yes
- What gets generated: `app/`, `public/`, `package.json`, `tsconfig.json`, `next.config.ts`
- Running `npm run dev` and opening `localhost:3000`
- The entry point: `app/page.tsx` — this is the home page
- Making a small change and seeing the hot reload

Transitions: The project is running. The next section digs into how this project is organized — what each folder is for and how to keep it clean.

Key Principles:

- `create-next-app` is the official and recommended way to start
- Hot reload means you see changes instantly — no manual refresh needed
- The folder structure is not arbitrary: Next.js reads it to build routing and pages

Examples:

- The setup wizard output: what "App Router" and "src/ directory" mean in plain English
- Editing `app/page.tsx` : change the `<h1>` text and watch the browser update immediately
- The `public/` folder: drop an image there and access it at `/my-image.png` — no import needed

Hands-On Exercise: Scaffold a new Next.js project with TypeScript and Tailwind enabled. Run the dev server. Edit the homepage `<h1>` to display your name. Add a paragraph below it describing what you're building. Confirm the page updates live in the browser.

Success Criteria:

- Project runs without errors on `localhost:3000`
 - Homepage shows custom name and description text
 - Changes reflect immediately without manual reload
-

Section 03: Project Structure and Organization

03.0 How a Next.js Project Is Organized

Learning Objectives:

- Identify the purpose of each top-level folder and file in a Next.js project
- Understand what belongs in `app/` , `components/` , `public/` , and `layouts/`
- Recognize which files are Next.js conventions and which are project conventions

Content Outline:

- Top-level folders and their roles:
 - `app/` — the routing and page layer (Next.js convention, required)
 - `public/` — static assets served directly (Next.js convention)
 - `components/` — reusable UI components (project convention, not enforced by Next.js)
 - `lib/` or `utils/` — helper functions and shared logic (mention only)
- Top-level files and their roles:
 - `next.config.ts` — framework configuration
 - `tsconfig.json` — TypeScript configuration
 - `package.json` — dependencies and scripts
 - `.env.local` — environment variables (connection to Day 14 memory bank concepts)
- The distinction between Next.js enforced conventions (inside `app/`) and team-defined conventions (everything else)

Transitions: Knowing where things live at the top level is the start. The next lesson focuses on the most important organizational decision: what's a page, what's a component, and what's a layout?

Key Principles:

- `app/` is sacred: Next.js reads it to build your routes — don't put random files there
- `components/` is yours to design: Next.js doesn't care, but your team does
- Clean project structure is a communication tool — it tells the next developer where to look

Examples:

- A `Navbar` component: lives in `components/Navbar.tsx` , not in `app/`
 - A product image: lives in `public/product.jpg` , accessible at `/product.jpg`
 - A helper function for formatting dates: lives in `lib/formatDate.ts` , not mixed into a component
-

03.1 Pages vs Components vs Layouts — Where Does What Live?

Learning Objectives:

- Define the distinction between a page, a component, and a layout in Next.js
- Know which folder each type belongs in
- Apply the right classification when creating a new file

Content Outline:

- **Pages (Views):** Files inside `app/` that map to a URL — `app/about/page.tsx` → `/about`
 - Named semantically: `HomePage`, `ContactPage`, `AboutPage`
 - They are full screens the user navigates to
- **Components (Widgets):** Reusable UI pieces in `components/` that have no URL
 - Named for what they represent: `Navbar`, `Button`, `ProductCard`
 - They render inside pages or other components
- **Layouts:** Wrapper components that define persistent structure (Navbar + Footer) around changing content
 - `app/layout.tsx` — the root layout wrapping every page
 - The `children` prop as the slot where page content goes
- The mental model: Layout wraps Pages; Pages use Components

Transitions: Now that we understand what type of file to create, the next lesson is a hands-on tour of navigating a real Next.js codebase.

Key Principles:

- If it has a URL, it's a page — put it in `app/`
- If it's reusable UI with no URL, it's a component — put it in `components/`
- Layouts prevent duplication: write the Navbar once, wrap every page in it

Examples:

- `/dashboard` is a page (`app/dashboard/page.tsx`); the sidebar inside it is a component (`components/Sidebar.tsx`)
- `app/layout.tsx` contains the `<Navbar />` and `<Footer />` so they appear on every page automatically
- Wrong: putting a `Button` component in `app/` — it's not a route, it doesn't belong there

03.2 Navigating a Real Codebase

Learning Objectives:

- Open an unfamiliar Next.js project and quickly locate key files
- Identify what a file does based on its location and name
- Add a new page and a new component to an existing project without breaking the structure

Content Outline:

- The skill of reading a codebase: start at `app/`, read `layout.tsx`, find `page.tsx`
- Using the file tree as a map: folder depth = route depth
- Spotting Next.js conventions vs custom project files
- Adding a new page: create `app/contact/page.tsx` with a basic export
- Adding a new component: create `components/Hero.tsx` and import it into a page

Transitions: We can now create pages and components with confidence. The next section covers the App Router in detail — the system that turns folder structure into navigation.

Key Principles:

- Read the structure before writing any code
- Folder = route segment in `app/`
- Every page file must export a default function

Examples:

- Opening a real Next.js project and naming what each file does before reading it
- Creating `app/projects/page.tsx` to add a `/projects` route
- Importing `<Hero />` from `components/Hero.tsx` into `app/page.tsx`

Hands-On Exercise: Given a provided starter Next.js project with a homepage and a layout, add: (1) a new `/about` page with a short paragraph, (2) a `<Footer />` component in `components/` with your name and the current year, (3) import and render `<Footer />` inside `app/layout.tsx`.

Success Criteria:

- `/about` route exists and renders the paragraph
 - `<Footer />` is in the correct folder and renders correctly
 - `<Footer />` appears on both the homepage and the about page via the layout
-

Section 04: App Router and Navigation

04.0 App Router Conventions and Top-Level Folders

Learning Objectives:

- Understand what the App Router is and how it differs from older Next.js routing
- Identify the top-level folder conventions inside `app/`
- Recognize reserved file names and what each one does

Content Outline:

- What the App Router is: Next.js 13+ system where folders define routes and special files define behavior
- Why it replaced the older `pages/` directory (mention only — students don't need to learn the old system)
- Top-level folder conventions inside `app/` :
 - Regular folders become route segments: `app/blog/` → `/blog`
 - Folders in parentheses `(group)` are route groups — they organize routes without affecting the URL
 - Folders with an underscore `_folder` are private — Next.js ignores them for routing
- The most important reserved files:
 - `page.tsx` — the UI for a route (required for a route to exist)
 - `layout.tsx` — shared UI wrapping a segment and its children
 - `loading.tsx` — loading UI shown while a page is fetching
 - `error.tsx` — error UI for a route segment
 - `not-found.tsx` — 404 UI for a segment

Transitions: With the folder layer understood, the next lesson covers the file conventions inside those folders — especially for nested routes and route groups.

Key Principles:

- In App Router, **the folder structure is the route structure**
- Reserved file names are Next.js contracts — they must be named exactly
- Route groups `(name)` let you organize files without changing URLs

Examples:

- `app/blog/page.tsx` → renders at `/blog`
 - `app/(marketing)/home/page.tsx` → renders at `/home` (the `(marketing)` group is invisible in the URL)
 - `app/dashboard/loading.tsx` → shows a spinner automatically while the dashboard page loads
-

04.1 Routing Files, Nested Routes, and Route Groups

Learning Objectives:

- Create nested routes using folder nesting in `app/`
- Explain what route groups are and when to use them
- Understand how layouts compose across nested routes

Content Outline:

- Nested routes: adding a folder inside a folder creates a child route
 - `app/blog/page.tsx` → `/blog`
 - `app/blog/[slug]/page.tsx` → `/blog/anything` (dynamic route — mention only, not deep dive)
- How layouts nest: `app/layout.tsx` wraps everything; `app/dashboard/layout.tsx` wraps only dashboard pages
- Route groups in practice:
 - `(auth)` group: contains `login/` and `register/` pages without `/auth` in the URL
 - `(marketing)` group: contains landing pages with their own layout, separate from app pages
- Private folders with `_`: useful for component folders colocated with routes

Transitions: Now that we understand how routing and nesting work through folders, the next lesson covers metadata — the file conventions that control SEO, page titles, and social sharing.

Key Principles:

- Nesting folders = nesting routes, and nesting layouts
- Route groups keep the URL clean while letting you organize files logically
- Each layout in a nested segment only wraps its own children

Examples:

- An `(app)` group with its own `layout.tsx` (with sidebar) vs a `(public)` group with a different `layout.tsx` (with top navbar)
 - `app/blog/[slug]/page.tsx`: the `[slug]` folder is a dynamic segment — a preview of what's coming in later lessons
 - A `_components/` folder inside `app/dashboard/` that contains components only used by the dashboard (Next.js ignores it for routing)
-

04.2 Metadata File Conventions

Learning Objectives:

- Understand what metadata is and why it matters for SEO and sharing
- Use the `metadata` export in `page.tsx` and `layout.tsx` to set page titles and descriptions
- Identify the special image files Next.js recognizes automatically

Content Outline:

- What metadata is: information about a page consumed by browsers, search engines, and social platforms (title, description, Open Graph)
- Two ways to define metadata in Next.js:
 - Static: export a `metadata` object from `page.tsx` or `layout.tsx`
 - Dynamic: export a `generateMetadata` function (mention only)
- The most useful metadata fields:
 - `title` — the browser tab title
 - `description` — the search engine snippet
 - `openGraph.title` and `openGraph.description` — for social sharing
- Special metadata files Next.js recognizes by filename:
 - `favicon.ico` — browser tab icon (drop in `app/`)
 - `opengraph-image.png` — social sharing preview image (drop in a route folder)
 - `robots.txt` — search engine crawl instructions (drop in `app/`)

- `sitemap.xml` — site map for search engines (mention only)

Transitions: Metadata handled. The next lesson puts the App Router to work — building a real multi-page structure and navigating between pages.

Key Principles:

- Metadata is not optional — it directly affects how the app appears in search and when shared
- Static metadata via `export const metadata` is the simplest approach and covers 90% of use cases
- Placing the right file in the right folder is all it takes for `favicon.ico` and `opengraph-image.png`

Examples:

- `export const metadata = { title: "About Us", description: "Learn more about our team" }` in `app/about/page.tsx`
 - Dropping `favicon.ico` into `app/` and seeing the browser tab icon update
 - A missing `description` in metadata: search engines generate their own, often poorly
-

04.3 Building a Multi-Page Structure with the App Router

Learning Objectives:

- Build a three-page Next.js app using App Router conventions
- Navigate between pages using the Next.js `<Link>` component
- Apply a shared layout wrapping all pages

Content Outline:

- Creating three routes: `/`, `/about`, `/projects`
- Writing a root `layout.tsx` with a `<Navbar />` using `<Link>` for navigation
- Using `<Link href="/about">About</Link>` instead of `<a href="/about">About</a>` — and why it matters for SPA behavior
- Adding static `metadata` to each page
- Adding a `favicon.ico`
- Testing navigation: pages swap without full browser refresh

Transitions: The app has structure and navigation. The next section introduces the rendering question: when should the browser handle the page, and when should the server?

Key Principles:

- Always use `<Link>` for internal navigation — never `<a>` for in-app routes
- The `<Link>` component keeps the SPA behavior: no full page reload
- The root `layout.tsx` is the single place to put your `<Navbar />` — not in every page

Examples:

- `<Link href="/about">About</Link>` in the Navbar — smooth SPA navigation
- `<a href="/about">About</a>` — causes a full page reload, loses SPA behavior (the anti-pattern)
- Adding `export const metadata = { title: "Projects" }` to the projects page

Hands-On Exercise: Build a three-page Next.js app: Home, About, and Projects. Add a shared `<Navbar />` with `<Link>` navigation in `app/layout.tsx`. Add static metadata with a unique title to each page. Add a `favicon.ico`. Verify navigation works without full page reloads.

Success Criteria:

- Three routes exist and render correctly
- `<Navbar />` is in the layout, not repeated in each page
- Navigation uses `<Link>`, not `<a>`
- Each page has a unique `<title>` visible in the browser tab

- `favicon.ico` appears in the browser tab
-

Section 05: Rendering Strategies

05.0 Client-Side Rendering (CSR) vs Server-Side Rendering (SSR)

Learning Objectives:

- Explain the difference between CSR and SSR in plain terms
- Identify the trade-offs of each approach
- Understand how Next.js handles both and when to use `"use client"`

Content Outline:

- **Client-Side Rendering (CSR):**
 - The browser downloads JavaScript, runs it, and builds the HTML
 - What the user sees first: a blank page or loading skeleton
 - When to use it: interactive UI, components that respond to user input
 - How to mark a component as CSR in Next.js: `"use client"` at the top of the file
- **Server-Side Rendering (SSR):**
 - The server generates the HTML and sends it fully formed to the browser
 - What the user sees first: complete content immediately
 - When to use it: pages that need to be indexed by search engines, or that display data fetched on load
 - In Next.js App Router: server components are the **default** — you only opt into client when needed
- The mental model: default to server, add `"use client"` only when the component needs interactivity
- Why this matters: performance, SEO, and the user's first impression

Transitions: Now that the theory is clear, the next lesson applies it: identifying which parts of a real UI should be server components and which need to be client components.

Key Principles:

- In Next.js App Router, all components are Server Components by default
- Add `"use client"` only when the component uses interactivity (state, events, browser APIs)
- SSR is better for SEO and perceived performance; CSR is necessary for interactivity

Examples:

- A blog post page: perfect for SSR — content is static, must be indexed by Google
 - A "Like" button: must be CSR — it responds to user clicks and needs state
 - An e-commerce product page: the product info is SSR; the "Add to Cart" button is CSR
-

05.1 Choosing the Right Rendering Strategy

Learning Objectives:

- Given a UI description, correctly classify each component as server or client
- Add `"use client"` to components that require it
- Recognize common mistakes when mixing server and client components

Content Outline:

- The decision rule: does this component need to respond to user input, use browser APIs, or manage state? → `"use client"`. Otherwise → server component
- What breaks when you use a client-only feature in a server component (and what the error looks like)
- The composition pattern: a server component can render a client component, but not the reverse
- Common UI elements and their correct rendering type:

- Navbar with links → server (no interactivity needed)
- Search bar with live filtering → client
- Product list from a database → server
- Dropdown menu → client
- Page title and metadata → server

Transitions: With rendering strategies understood, we move to the final content section: actually building a component with React's core primitives — props, state, and effects — at a first-look level.

Key Principles:

- Server first, client only when needed
- `"use client"` marks the boundary: everything in that file and its children runs in the browser
- You cannot import a Server Component into a Client Component

Examples:

- Adding `"use client"` to a `Counter` component that uses `useState` — without it, Next.js throws an error
- A `<ProductList />` that fetches data on the server and passes it to a `<ProductCard />` that handles a client-side "favorite" toggle
- The error message when you try to use `useState` without `"use client"`: reading and understanding it

Hands-On Exercise: Take a provided Next.js page with four components: a static `<PageTitle />`, a `<ContactForm />` with an input field, a `<VisitorCounter />` that displays a number, and a `<FooterLinks />` with navigation links. Classify each as server or client. Add `"use client"` only where needed. Confirm the page renders without errors.

Success Criteria:

- Only components that require interactivity have `"use client"`
- Static components remain server components
- No console errors related to client/server boundaries

Section 06: Your First Real Component in Next.js

06.0 Props, State, and Effects — A First Look (React 18 Standard)

Learning Objectives:

- Understand what props are and why components need them
- Understand what state is and why components need it
- Understand what effects are and what kind of problem they solve
- Know that these three concepts will be covered in full depth in the next lesson

Content Outline:

- **Props:** How a parent component passes data to a child
 - Like function parameters: the parent provides values, the child uses them
 - Props make components flexible and reusable
 - Example: `<UserCard name="Ana" role="Admin" />` — `name` and `role` are props
- **State (`useState`):** A value that belongs to a component and can change over time
 - When state changes, React re-renders the component automatically
 - Example: a counter that increments when you click a button — the count is state
 - `const [count, setCount] = useState(0)` — just the shape, not the deep dive
- **Effects (`useEffect`):** Running code in response to something happening (component appearing, data changing)
 - Example: fetching data when a component loads for the first time
 - `useEffect(() => { ... }, [])` — just the shape, not the deep dive
- The React 18 hook standard: `useState`, `useEffect`, and custom hooks are the modern way (no class components)

- Emphasis: **Full mastery of these comes in the next lesson** — today is a first look

Transitions: Now that we've seen the three pillars of React component behavior, the next lesson applies props directly — building a reusable component that accepts data from its parent.

Key Principles:

- Props flow down: parent → child (never the reverse)
- State lives inside a component and triggers re-renders when it changes
- Effects are for side effects: fetching data, subscriptions, timers — not for calculating values
- These three concepts (props, state, effects) are the foundation of everything in React

Examples:

- Props: `<Button label="Save" color="blue" />` — the same `<Button />` component used differently in two places
 - State: a toggle switch — clicked = on, clicked again = off. The on/off value is state
 - Effect: "when this page loads, fetch the user's profile from the API"
-

06.1 Building a Component with Props

Learning Objectives:

- Define a component that accepts props with TypeScript types
- Use props to render dynamic content inside a component
- Render the same component multiple times with different prop values

Content Outline:

- Defining a TypeScript interface for a component's props
- Accessing props inside the component: `const Card = ({ title, description }: CardProps)`
- Rendering dynamic content using props: `<h2>{title}</h2>`
- Using the same component multiple times with different data
- Default prop values: `description = "No description provided"`

Transitions: We've now written real React components with real data flow. The final lesson of this section shows what happens when developers skip the patterns we've learned.

Key Principles:

- Type your props with TypeScript interfaces — it prevents bugs and self-documents your component
- Destructure props in the function signature for cleaner code
- Reusability is the point: if you're writing the same JSX twice, make it a component with props

Examples:

- `<TeamMemberCard name="Laura" role="Engineer" />` and `<TeamMemberCard name="Pedro" role="Designer" />` — same component, different data
- A TypeScript interface: `interface TeamMemberCardProps { name: string; role: string }`
- Using a default value: `<AlertBanner message="Something went wrong" type="error" />` where `type` has a default of `"info"`

Hands-On Exercise: Build a `<ProjectCard />` component that accepts `title`, `description`, and `techStack` (an array of strings) as props. Type the props with a TypeScript interface. Render it three times inside the home page with different project data. Add a default value for `description`.

Success Criteria:

- `<ProjectCard />` accepts and renders all three props
- Props are typed with a TypeScript interface
- Component is rendered three times with distinct data

- Default `description` renders correctly when omitted
-

06.2 Anti-Patterns: Common React Beginner Mistakes

Learning Objectives:

- Identify the most common structural mistakes made when starting with React and Next.js
- Understand why each anti-pattern is problematic
- Know the correct approach for each

Content Outline:

Anti-pattern 1: Using `<a>` instead of `<Link>` for internal navigation

- What it looks like: `<a href="/about">About</a>` inside a Next.js app
- Why it's a problem: causes a full page reload, destroys the SPA behavior, resets client-side state
- Correct approach: `<Link href="/about">About</Link>` always for internal routes

Anti-pattern 2: Putting everything in one file

- What it looks like: 300 lines in `app/page.tsx` with the Navbar, Hero, three sections, and Footer all defined inline
- Why it's a problem: impossible to maintain, nothing is reusable, every change risks breaking the whole page
- Correct approach: each component in its own file in `components/`, imported where needed

Anti-pattern 3: Putting non-route files in `app/`

- What it looks like: `app/Button.tsx`, `app/helpers.ts`, `app/styles.css`
- Why it's a problem: pollutes the routing layer, creates confusion, may cause unexpected Next.js behavior
- Correct approach: components in `components/`, utilities in `lib/` or `utils/`, `app/` is for routes only

Anti-pattern 4: Using Class Components

- What it looks like: `class MyComponent extends React.Component { render() { ... } }`
- Why it's a problem: outdated since React 16.8, AI tools trained on older data may generate them, they cannot use hooks
- Correct approach: always use functional components with hooks

Anti-pattern 5: Direct DOM manipulation

- What it looks like: `document.getElementById("title").innerText = "Hello"` inside a React component
- Why it's a problem: bypasses React's rendering cycle, causes unpredictable UI bugs
- Correct approach: use state to drive UI changes — let React update the DOM

Transitions: With the correct patterns internalized and the anti-patterns named, the next section checks your understanding before the final challenge.

Key Principles:

- `<Link>` is non-negotiable for internal navigation in Next.js
- `app/` is the routing layer — only route files belong there
- Functional components with hooks are the modern React standard — if AI generates class components, correct them

Examples:

- AI-generated code with `<a href="/contact">` — spot it, fix it
 - A 400-line page file with everything inlined — how to refactor it
 - A `class Hero extends React.Component` — what it means and how to rewrite it as a function
-

Section 07: Assessment

07.0 Practice: Assemble a Simple Next.js Page from Components

Learning Objectives:

- Apply the complete skill set from this course in a single integrated exercise
- Make architectural decisions independently: what's a page, what's a component, what's in the layout
- Identify and fix structural and navigation errors in provided code

Content Outline:

- Given a design mockup (described in text), build the page from scratch:
 - A root layout with `<Navbar />` and `<Footer />`
 - A home page with three sections: Hero, Feature List, and Contact CTA
 - Each section as its own component in `components/`
 - Correct file locations, naming conventions, and `<Link>` usage throughout
- A second task: review a provided snippet of broken Next.js code and identify all anti-patterns
- Categorize each component as server or client

Transitions: After this practice, the knowledge check consolidates and verifies understanding before the final challenge.

Key Principles:

- Architecture first: plan where each file lives before writing any code
- Review your own work against the quality checklist: are all conventions followed?

Examples:

- The feature list: is it a component? Yes — it's reusable UI
- The `<Link>` in `<Navbar />`: server component (no interactivity), `"use client"` not needed
- Spotting `<a href="/about">` in provided code and correcting it

Hands-On Exercise: Build a landing page for a fictional AI consulting company. Requirements: root layout with `<Navbar />` (links to `/`, `/services`, `/contact`) and `<Footer />`; home page with three separate components (`<Hero />`, `<ServiceList />`, `<ContactCTA />`); an `/about` page; correct metadata on each page; all components in `components/`; no `<a>` for internal links.

Success Criteria:

- All routes render correctly
- Component files are in `components/`, not in `app/`
- All internal navigation uses `<Link>`
- Layout wraps all pages (Navbar and Footer appear everywhere)
- Each page has a unique title via `metadata`

07.1 Knowledge Check

Learning Objectives:

- Verify conceptual understanding of React components, Next.js structure, App Router, and rendering strategies
- Confirm students can identify correct and incorrect approaches
- Prepare students for the final code challenge

Format: Multiple choice (6 questions)

Questions:

Q1. You are building a multi-page Next.js application. You want a `<Navbar />` to appear on every page without repeating it in each page file. Where should you place it?

- A) In every `page.tsx` file directly
- B) In `app/layout.tsx` 

- C) In `public/layout.tsx`
- D) In `components/layout.tsx`

Q2. You create a file at `app/dashboard/settings/page.tsx`. What URL does this file map to?

- A) `/app/dashboard/settings`
- B) `/page`
- C) `/dashboard/settings` ✓
- D) `/settings`

Q3. Which of the following correctly uses Next.js navigation inside a component?

- A) `<a href="/about">About</a>`
- B) `<navigate to="/about">About</navigate>`
- C) `<Link href="/about">About</Link>` ✓
- D) `<router-link to="/about">About</router-link>`

Q4. A `<ProductList />` component fetches data from a database and renders a static list — no user interaction needed. Which rendering approach is most appropriate?

- A) Add `"use client"` at the top — all data fetching requires it
- B) Leave it as a Server Component (the default) ✓
- C) Add `"use server"` at the top
- D) Wrap it in `React.memo()`

Q5. A developer writes a `<Navbar />` component and puts the file at `app/Navbar.tsx`. What is wrong with this?

- A) The component name must be lowercase
- B) Navbar components must be in `public/`
- C) Non-route files should not be placed in `app/` — it should be in `components/Navbar.tsx` ✓
- D) Nothing is wrong — this is the correct location

Q6. You are reviewing AI-generated code and see `class HeroSection extends React.Component`. What should you do?

- A) Use it as-is — class components are the modern React standard
- B) Rewrite it as a functional component, since class components are outdated and cannot use hooks ✓
- C) Add `"use client"` to make it work
- D) Move it to `app/` to fix the class component error

Evaluation Criteria:

- 6/6: Full mastery — ready for Skill 12.2
- 4–5/6: Solid understanding with minor gaps — review flagged lessons before proceeding
- 0–3/6: Foundational gaps — revisit Sections 01–05 before the code challenge

Section 08: Code Challenge

08.0 Build a Multi-Section Personal Portfolio in Next.js 🚀

Scenario: You're building a personal portfolio site to showcase your work as an AI Engineer. The site needs multiple pages, a consistent layout, and be built entirely from reusable components using Next.js App Router conventions.

Requirements:

1. Scaffold a Next.js project with TypeScript and Tailwind enabled
2. Create a root layout (`app/layout.tsx`) with a `<Navbar />` component linking to all pages and a `<Footer />` component — both placed in `components/`
3. Build three pages using App Router file conventions:

- `/` — Home page with a `<Hero />` component and a `<SkillList />` component
 - `/projects` — Projects page with at least three `<ProjectCard />` components, each receiving `title`, `description`, and `techStack` as props
 - `/contact` — Contact page with a `<ContactInfo />` component
4. All reusable components must live in `components/` — none in `app/`
 5. All internal navigation must use `<Link>`, never `<a>`
 6. Each page must export a `metadata` object with a unique `title` and `description`
 7. Add a `favicon.ico` in the `app/` directory
 8. Classify each component correctly: add `"use client"` only to components that require it (if any)
 9. No class components — all components must be functional

Constraints:

- TypeScript only (`.tsx` files)
- Tailwind for styling — no inline styles or external CSS files
- No additional libraries beyond what `create-next-app` installs

Expected Output / Behavior:

- Running `npm run dev` produces no errors
- All three routes render correctly in the browser
- `<Navbar />` and `<Footer />` appear on every page
- Navigating between pages does not cause a full browser reload
- Each page shows a distinct title in the browser tab
- `<ProjectCard />` renders three times with different data on the `/projects` page